

M10：进制、位运算、地址空间、sizeof、函数指针与格式化

CSC3002 基础讲义 · 第 6 册 · 前置：M1-M9 的基本类型、表达式、函数、数组、指针与流

核心问题：怎样从表示方式、类型和单位出发，把零散题变成可以检查步骤的计算？ 学完应能手算二进制与十六进制，按位操作无符号值，写清地址空间的计量条件，区分数组大小与指针大小，调用函数指针并控制输出格式。

本册按实际课件 L6 的表示/内存部分、L5 的格式部分以及 E22 Q14、Q17、Q20 校准，额外用 Q11 复习表达式基础。采用 C++17；题设给出位宽、寻址单位时按题设推导，未给出的大小不擅自补定。最后列出旧卷尚需后续模块处理的题目，避免把“六册基础讲完”等同于“全卷覆盖”。

阅读依据与课程定位

这是对实际课程材料重新核对后的修订版，核对日期为 2026-10-08。L 后的数字是课件代号；“第几张”按文件实际幻灯片顺序计数，从封面开始，不沿用旧模板页脚。E22 页码为 PDF 页码。

- L1: 1-CSC3002_Lecture Notes Overview of C++(5).pptx
- L5: 5-CSC3002_Lecture Notes Streams(3).pptx
- L6: 6-CSC3002_Pointers and Arrays(4).pptx
- E22: CSC3002 Final 2022 Spring.pdf

本册学习任务	原材料定位	要能独立完成的动作
进制与表示	L6 第 3-7 张	由位权转换，同一数值用不同记法
字节、word、地址长度与容量	L6 第 8-16 张；E22 Q14	地址个数乘寻址单位，再换算容量
按位操作与颜色分量	E22 Q17；L1 表达式部分为前置	右移目标字节，再用掩码保留低位
函数指针	L6 第 34 张；E22 Q20	从名字向外读声明，区分返回指针的函数
格式化、精度和宽度	L5 第 6-15 张；E22 Q31/Q32 的格式要求	维护流状态，辨认数值与显示形式
数值表达式复习	L1 第 34-48 张；E22 Q11	每一步写类型、分组与中间值

课件决定本册的概念范围；旧卷用于练题型和深度，不预测今年题目或分值。本册的解释、状态表和答案由教学推导得到；未找到官方答案册，因此不称为官方标准答案。保留的最小例子与一般练习属于自编教学补充；涉及原材料时逐项标明“课件例题”“原题代码”或“题意转述”。

当前 L1 第 2 张写的是闭卷，E22 第 1 页写的是可带一张双面 A4 的限制性开卷，两者不能混作今年规则。六册只覆盖 C++ 基础这一段，旧卷的数据结构、算法和部分继承多态内容仍需后续学习。

第一部分：数值不变，表示方式可以改变

1. 二进制、十六进制是什么？

进制规定每一位的权重以及每一位能使用的数字。 十进制逢十进一；二进制只使用 0、1，每位权重为 2 的幂；十六进制使用 0-9、A-F，每位权重为 16 的幂。

例如二进制 101101：

```

位权：32  16   8   4   2   1
数字： 1   0   1   1   0   1
取和：32 + 8 + 4 + 1 = 45

```

同一个数可以写为十进制 45、二进制 101101、十六进制 2D。表示方式改变，不是把值改成另一种东西。

2. 为什么一个十六进制位对应四个二进制位？

因为 $16 = 2$ 的四次方。四位二进制能表示 0 到 15，恰好对应十六进制一个数字。

十六进制	二进制	十进制
0	0000	0
5	0101	5
A	1010	10
D	1101	13
F	1111	15

把 101101 从右向左每四位分组，左边补零成 0010 1101，得到 2D。把 0x3A 反过来展开为 0011 1010，再相加得到 $32+16+8+2=58$ 。

从十进制转二进制也可连续除以 2 记录余数，再倒序读；转十六进制则除以 16。45 除以 16 得商 2 余 13，13 写 D，所以为 2D。

3. C++ 怎么写这些字面量？

```

int a = 45;
int b = 0b101101;
int c = 0x2D;
unsigned int d = 45u;

```

0b 表示二进制前缀，0x 表示十六进制前缀，u 表示无符号整数后缀。这些小数值的前三个对象都得到 45。默认 cout 输出整数通常按十进制，不会记住源代码原来写的是 0x。

小心前导零：整数 010 是八进制字面量，值为十进制 8，不是十进制 10。普通十进制整数不要随意补前导零。见 [整数字面量规则](#)。

4. 位宽和数值范围怎样连接？

若有 n 个二进制位，能形成 2 的 n 次方种组合。 n 位无符号整数范围是 0 到 2 的 n 次方减 1；8 位无符号数范围为 0-255。

若题目明确采用 n 位二进制补码有符号表示，范围为 -2 的 $n-1$ 次方到 2 的 $n-1$ 次方减 1。8 位补码范围为 $-128-127$ 。

本讲以 C++17 为基线，不能把所有机器的有符号表示都不加条件认定为补码；课程题明确给定补码时再按此计算。常见当前平台采用补码，不代表可以省略题设检查。

8 位补码 -5 可用 $256-5=251$ 表示，位模式为 11111011，即 0xFB。这个位模式按无符号解释是 251，按给定补码有符号解释是 -5 。**相同比特怎样解释，依赖类型与表示约定。**

第二部分：位运算是逐位操作

5. &、|、^、~ 分别干什么？

在整数位运算语境中，& 是逐位与，| 是逐位或，^ 是逐位异或，~ 是逐位取反。它们处理位模式，而不是只把整项变成一个真假值。

输入位 a	输入位 b	a 与 b	a 或 b	a 异或 b
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

例如只显示低四位，10 为 1010，12 为 1100：

```
1010 & 1100 = 1000 = 8
1010 | 1100 = 1110 = 14
1010 ^ 1100 = 0110 = 6
```

~10u 则翻转 unsigned int 的全部位，不是只翻四位。若只想保留低四位结果，使用 (~10u) & 0xFu，得到 5。图画了几位与类型实际有几位不能混淆。

6. 位运算和逻辑运算不能互换

```
int a = 2, b = 4;
int bits = a & b;
bool truth = a && b;
```

2 的低位为 0010，4 为 0100，没有共同的 1 位，因此 bits=0。逻辑 && 则把两个非零值视为真，所以 truth=true。

逻辑 && 和 || 还具有短路规则：左边已经决定结果时不求值右边；内建整数位运算 &、| 没有这种短路。p != nullptr && *p == 5 可先检查空指针；不能把 && 换成 & 来保持同样的保护语义。

同一个 & 出现在 &x 时是取地址，出现在 int& r 时属于引用声明，出现在 a & b 时是整数按位与。先判断语境，再讲符号含义。

7. 位掩码是什么，为什么要用它？

掩码是用来选择某些位的位模式。 若一个标志位对应第 k 位，可用 1u << k 生成只在该位为 1 的掩码，这里位编号从最低位 0 开始。

```
unsigned int flags = 0;
unsigned int mask = 1u << 2; // 第 2 位，值为 4
flags |= mask;               // 设为 1
bool on = (flags & mask) != 0;
flags ^= mask;               // 翻转
flags &= ~mask;              // 清为 0
```

|=、^=、&= 分别把相应逐位结果写回左对象。检查用 & 保留目标位，设位用 |，清位用与取反掩码，翻位用 ^。这些操作的理由来自真值表，避免只记符号。

例如 flags=1010、mask=0100：设位得到 1110；清位得到 1010；再用相同掩码异或得到 1110。每步都应从当前值开始，不能混用旧状态。

8. 为什么检查掩码要给 & 加括号？

应写 (flags & mask) != 0。若写成 flags & mask != 0，比较 != 结合更紧，会按 flags & (mask != 0) 解析，不再是预期的位检查。

例如 flags=2、mask=2：正确写法计算 2&2=2，再比较不等于 0，结果真；错误分组先得 mask!=0 为 true，再作 2&1=0，结果假。这个例子说明：先解析表达式结构，再计算结果，与 M5 的指针自增读法相同。

9. 左移右移怎样手推？

本节只用位数和取值条件明确的无符号整数，避免把不同版本的有符号移位规则混在一起。

```
unsigned int x = 5u;
unsigned int a = x << 2;
unsigned int b = 20u >> 2;
```

5 的低位 0101 左移两位成为 010100，得到 20；20 右移两位得到 5。对无符号值，移出的高位丢弃，空出的位填零，结果受目标类型位宽限制。

移位数量必须非负，并小于**经整数提升后左操作数类型的位宽**。若题设 unsigned int 为 32 位，1u << 31 在范围内，1u << 32 不是合法的该类移位，不能把它当成结果 0。

小整数类型可能先被提升：常见八位 unsigned char 参与 ~、<< 等运算时，结果并不自动只有八位。计算位宽前先判断表达式类型。移位有效性条件见 [移位运算](#)。

第三部分：地址空间先写单位，再算容量

10. n 位地址能区分多少个位置？

若题目规定有效地址宽度为 n 位，且每种位模式代表一个不同可寻址单元，则有 2 的 n 次方个地址，编号从 0 到 2 的 n 次方减 1。

若进一步规定**按字节寻址**，可寻址容量为 2 的 n 次方字节。若每个地址表示一个 4 字节字，则容量应再乘 4，而不是仍答字节寻址结果。

32 位字节地址可区分 2^{32} 个字节位置，为 4 GiB。这里说的是给定地址模型的空间大小，不是计算机一定安装了 4 GiB 物理内存，也不是进程可以无条件使用全部空间。

11. 位、字节、KiB、MiB、GiB 先统一

单位	本册换算约定
bit	一个二进制位
byte	C++ 字节；本节数值题另假定为 8 bit
KiB	2^{10} byte = 1024 byte
MiB	2^{20} byte
GiB	2^{30} byte

题目写 KB、MB 时，注意它采用十进制还是二进制定约；本册用 KiB 等名称表达二进制倍数，避免把 1000 与 1024 混用。

若地址宽度为 20 位、每地址 1 字节，则空间为 2^{20} 字节，即 1 MiB。若每地址 4 字节，则为 4 MiB；若只问地址个数，答案仍是 2^{20} 个，别把个数和容量混成同一单位。

12. 从容量反推至少多少地址位

一个连续空间包含 4096 个可寻址单元，要能区分这些位置，至少需要 12 位，因为 $2^{12}=4096$ 。若有 5000 个单元，12 位不够，至少 13 位，因为 $4096 < 5000 \leq 8192$ 。

注意 4096 个位置的最大编号是 4095，不能因为最后一个编号少 1 就少算一位。地址空间问题常错在上界编号、总数量和容量单位的互换。

第四部分：sizeof 问的是类型大小

13. sizeof 是什么？不是求字符串长度

sizeof 运算符提供相应对象类型占用的字节数，结果类型为 `std::size_t`。 可以写 `sizeof(int)`，也可对表达式写 `sizeof x` 或 `sizeof(x)`。

```
char text[8] = "abc";
int a[4] = {1, 2, 3, 4};
int* p = a;
```

`sizeof(text)=8`；`sizeof(a)=4*sizeof(int)`；`sizeof(p)=sizeof(int*)`。字符串长度 3、数组元素数 4、指针对象字节数是三种问题。

`sizeof(char)` 必为 1；`sizeof(int)`、`sizeof(bool)`、`sizeof(指针)` 等应依实现或题设确定。不能仅凭“64 位系统”就说所有整数都是 8 字节。见 [sizeof 规则](#)。

14. sizeof 为什么不把数组总是退化成指针？

对实际数组表达式作 `sizeof`，不进行通常的数组到指针转换，因此保留数组大小信息。

```
int a[4]{};
auto count = sizeof(a) / sizeof(a[0]);
```

分子是四个 `int` 的总字节数，分母是一个 `int` 的字节数，比例为 4。这里 `auto` 推断结果类型；不能把这个公式无条件放到数组形参上。

```
void inspect(int a[]) {
    // 参数已调整为 int*, sizeof(a) 量指针
}
```

在 inspect 内看见 [] 并不表示参数仍保留调用者数组长度。M3 已说明这种参数调整，sizeof 只是把差别暴露出来。

15. sizeof 一般不求值，也不量“引用外壳”

```
int n = 5;
auto bytes = sizeof(n++);
int& r = n;
```

标准 C++ 的这个 sizeof 操作数不求值，因此 n 仍为 5；bytes 为 sizeof(int)。表达式里出现 ++，不意味着一定执行了自增。

sizeof(r) 得到被引用整数类型的大小，不提供编译器实现引用可能用到的隐藏存储大小。

sizeof(std::string) 也量 string 对象本身的表示，不是其当前字符序列长度。

对类类型，sizeof 还包括布局填充。struct S { char c; int n; }; 的大小不能简单保证为 sizeof(char)+sizeof(int)；若题目要数字，需要给定布局和对齐规则或测量具体实现。

第五部分：函数也可以通过指针调用

16. 函数指针是什么？先区别调用结果与函数位置

函数指针保存可用于调用某个具有相应函数类型的函数的指针值。 它不是调用结果，也不是保存一个 int 对象的指针。

```
int add(int a, int b) { return a + b; }
int multiply(int a, int b) { return a * b; }
int (*op)(int, int) = &add;
```

从名字向外读：(*op) 表示 op 是指针；后面的 (int,int) 指向的函数接收两个 int；最前面的 int 是返回类型。括号必不可少。

声明	意义
int (*op)(int, int)	指向“两 int 参数、返回 int”的函数的指针
int* f(int)	一个函数，接收 int，返回 int*

op(3,4) 或 (*op)(3,4) 调用当前目标函数，得到 7。这里函数名在适当语境也可转换为函数指针，因此初始化可写 = add。

17. 改变函数指针，为什么调用结果改变？

```
op = &multiply;
int value = op(3, 4);
```

指针对象 op 保存的目标变为 multiply，接下来提供相同实参 3、4，得到 12。改变的是所选函数，不是把 add 的实现改写为乘法。

参数与返回类型要兼容。指向 `int(int,int)` 的指针不能直接拿一个不兼容的 `double(double)` 函数来正常赋值调用。空函数指针也不能直接调用。

实际用途是把操作作为参数交给另一个函数：

```
int apply(int x, int y, int (*operation)(int, int)) {
    return operation(x, y);
}
```

`apply` 接收数值和函数指针，再调用该操作。`apply(3,4,add)` 为 7，`apply(3,4,multiply)` 为 12。本例约定 `operation` 非空，不引入额外泛型工具。

第六部分：输出格式也是状态

18. hex、dec 改的是显示方式

```
int n = 45;
std::cout << std::hex << n << ' '
          << std::dec << n << '\n';
```

输出 `2d 45`。`std::hex` 让后续整数按十六进制输出，`std::dec` 改回十进制；`n` 的数值始终是 45。默认十六进制字母为小写，其他标志可改变显示。

这类格式设置通常会保留在流中，直到再改变。若只想让某一个数显示十六进制，随后记得恢复所需格式，否则后面的输出也可能受影响。

19. setw、setfill、fixed、setprecision 分别控制什么？

使用 `<iomanip>`：

```
std::cout << std::setfill('0') << std::setw(4)
          << 7 << ' ' << 8 << '\n';
std::cout << std::fixed << std::setprecision(2)
          << 3.5 << '\n';
```

第一行输出 `0007 8`：`setw(4)` 设置下一次相应格式化输出的最小宽度，数字不够宽时用当前填充字符补齐；它不把所有后续整数永久变成四位。`setfill` 的填充设置则会继续保留。

第二行输出 `3.50`：`fixed` 选定固定小数形式，此时 `setprecision(2)` 控制小数点后两位。在默认浮点格式下，`precision` 通常控制有效数字数量，不能把“precision 永远是小数位数”当通用规则。

宽度是最小宽度，不是截断长度；一个本来有五位的整数不会因 `setw(4)` 被截成四位。可查 [参数化流操纵器](#)。

20. 如果旧题出现 printf，先按类型匹配格式

包含 `<cstdio>`，可用 `std::printf`：

```
int n = 45;
std::printf("%d %x %.2f\n", n,
            static_cast<unsigned int>(n), 3.5);
```

`%d` 对应 `int` 的十进制输出；`%x` 对应 `unsigned int` 的十六进制输出；`%.2f` 在此输出 `double` 的两位小数；字符串里的 `\n` 为换行。输出 `45 2d 3.50`。

printf 使用可变参数，格式与实际参数类型不匹配可能导致未定义行为，不能认为它会像流重载那样自动按真实类型选择。打印 sizeof 结果可使用 %zu，因为其类型为 size_t。E22 没有单独的 printf 格式题；本节依据 L5 第 6-9 张补课件接口，不把它说成已见旧卷原题。

21. 完整程序：不同知识点回到同一套读法

```
#include <iomanip>
#include <iostream>

int add(int a, int b) { return a + b; }
int multiply(int a, int b) { return a * b; }

int main() {
    unsigned int flags = 0b1010u;
    flags |= 1u << 2;
    int a[4]{};
    int (*op)(int, int) = add;
    std::cout << std::hex << flags << ' '
              << std::dec << op(3, 4) << '\n';
    op = multiply;
    std::cout << op(3, 4) << ' '
              << sizeof(a) / sizeof(a[0]) << '\n';
    std::cout << std::fixed << std::setprecision(2)
              << 3.5 << '\n';
}
```

第一行 flags 设位后为 14，十六进制显示 e；函数指针调用 add 得 7，因此输出 e 7。第二行切换到 multiply 得 12，数组元素数为 4，输出 12 4。第三行输出 3.50。

回到 lecture notes：用课件数字校准方法

A. L6 第 3-7 张：同一个 42 有多种表示

课件使用 42 串起各进制：二进制 00101010，八进制 52，十进制 42，十六进制 2A。二进制的有效位权是 32、8、2，相加为 42；十六进制为 $2 \times 16 + 10 = 42$ 。

八进制每位对应三个二进制位，因为 $8 = 2^3$ ；十六进制每位对应四位，因为 $16 = 2^4$ 。不要把 52（八进制）读成五十二后再计算；先辨记法再还原数值。

第 7 张练习的 10001（二进制）为 $16 + 1 = 17$ ；177（八进制）为 $64 + 56 + 7 = 127$ ；AD（十六进制）为 $10 \times 16 + 13 = 173$ 。课件给出的十六位模式 1100 1010 1111 1110 每组映射一个十六进制位，得到 CAFE。

B. 地址数量、最大地址与物理内存要分开

L6 第 11 张讨论 N 位有效地址、字节寻址时的理论空间。有 2^N 个地址，最大编号是 $2^N - 1$ 。从 0 开始计数时，末尾编号比数量小 1。

因此 32 位地址的理论字节容量为 4,294,967,296 字节，最大地址编号才是 4,294,967,295。课件对应行写出的十进制容量少了 1，本册按“数量”修正。现实的物理内存、CPU 支持的地址位数和虚拟地址规则还受实现条件限制；不是一台机器名称带“64 位”就必然安装 2^{64} 字节 RAM。

L6 第 16 张的常见类型大小表也不是语言的无条件保证。数组占元素数量乘元素大小；结构体可能有 padding（填充）以满足对齐，不能总把字段大小直接相加；指针大小要按题设或具体平台。sizeof 测的是 C++ 字节数，题目给 8 位字节时再换算为比特。

C. L6 第 34 张 compare: 把函数当作这次选择的操作

下面保留课件的函数参数顺序与机制，整理成可读代码：

```
int compare(int (*fn)(int, int), int xx, int yy) {
    return fn(xx, yy);
}
int larger(int x, int y) { return x > y ? x : y; }
int smaller(int x, int y) { return x > y ? y : x; }
```

fn 是指针参数，目标函数接收两个 int 并返回 int。xx、yy 是数字参数。调用 compare(larger,10,5) 选择 larger，因此返回 10；换成 smaller，数字不变但算法变了，返回 5。

声明中的 (*fn) 让 fn 成为指针；后面的 (int,int) 说明所指函数的参数；最前面的 int 是返回类型。这与 M3 的指针对象一脉相承，只是目标从数据对象换成函数，不能再 *fn 当整数元素读写。

D. L5 第 12-15 张: precision 由浮点格式决定

默认浮点格式下 setprecision(k) 主要控制有效数字个数；fixed 与 scientific 下，它控制小数点后的位数。scientific 还把数量级放到指数里。

对新流和数值 314.159265，setprecision(6) 的默认形式为 314.159；fixed 加相同精度得到 314.159265；scientific 加 setprecision(3) 得到 3.142e+02，uppercase 后为 3.142E+02。指数位数在某些系统的显示可不同，课件第 13 张也提醒可能出现三位指数；数值意义相同。

setw 控制下一次相应插入的最小宽度；left/right 决定不足宽时在哪边补字符；setfill 设置补什么。课件示例 fixed、setprecision(4)、setw(10)、setfill('0') 输出 314.159265 时，四位小数为 314.1593，占 8 个字符，默认右对齐补两枚零，得到 00314.1593；若 left，则补在右边，得到 314.159300。

具体 double 通常不能精确存储任意十进制小数，所以 setprecision(20) 也不能凭空创造原本不存在的精度。它改变显示，不改变对象中已保存的近似值。复习格式题既要维护流状态，也要知道输入值的表示边界。

练习

题 1：进制转换

将二进制 11010110 转成十六进制和十进制；再说明 C++ 的 010 和 0x10 分别是十进制多少。

题 2：逐位操作

只观察低四位， $a=10u$ ， $b=12u$ 。求 $a\&b$ 、 $a|b$ 、 a^b 、 $(\sim a)\&0xFu$ ，并说明为何不能直接说 $\sim a=5$ 。

题 3：掩码与移位

flags 初值为 $0b0010u$ 。设置第 3 位，再清除第 1 位，最后翻转第 0 位，结果是多少？这里位编号从 0 开始。

题 4：地址空间

有 24 位有效地址，若每地址表示 1 字节，空间是多少 MiB？若每地址表示 4 字节呢？至少用多少地址位区分 5000 个位置？

题 5：sizeof

假设 int 为 4 字节、int* 为 8 字节。int a[5]{}; int* p=a; int& r=a[0]; 的 sizeof(a)、sizeof(p)、sizeof(r) 分别是多少？另执行 int n=2; auto s=sizeof(n++);, n 会变吗？

题 6：函数指针

int (*op)(int,int)=add; 后 op(2,5) 得什么？改为 op=multiply; 后呢？说明此指针类型里的两个括号各有什么作用。

题 7：流格式

从默认流状态开始，输出 std::hex << 26 << ' ' << 16 << ' ' << std::dec << 16, 显示什么？另一个新流输出 std::setfill('0') << std::setw(3) << 7 << ' ' << 8, 显示什么？

题 8：判断成立条件

“sizeof(bool) 一定为 1” “所有指针一定为 8 字节” “32 位地址一定代表 4 GiB 物理内存” “C++17 的 $1u < 32$ 在 32 位 unsigned int 下结果为 0”。哪些可直接当作无条件正确结论？

练习详细解析

题 1 解析

从右每四位分组：1101 0110，得到 D6。十进制为 $13 \times 16 + 6 = 214$ 。C++ 中 010 为八进制 10，即十进制 8；0x10 为十六进制 10，即十进制 16。字面量前缀决定如何解释数字，不能当作普通装饰。

题 2 解析

1010 与 1100 逐位操作：& 得 1000=8；| 得 1110=14；^ 得 0110=6。~ 翻转整个 unsigned int 位宽；再与低四位掩码 1111 相与，才只保留 0101=5。错误的 ~a=5 把示意图位数误当成类型实际位数。

题 3 解析

初值 0010；设第 3 位，与 1000 作或得到 1010；清第 1 位，与该位掩码的取反相与得到 1000；翻第 0 位，与 0001 异或得到 1001，即 9。每步记录当前 flags，不能都从 0010 独立计算。

题 4 解析

地址数量为 2^{24} 。字节寻址时为 2^{24} 字节=16 MiB；每地址 4 字节时再乘 4，得到 64 MiB。区分 5000 个位置需要 13 位，因为 $2^{12}=4096$ 不足， $2^{13}=8192$ 足够。先写“每地址多少字节”，再把地址个数换成容量。

题 5 解析

按题设，数组五个 int 为 20 字节；p 为 8 字节；r 的 sizeof 是被引用 int 类型大小，为 4。sizeof(n++) 不执行自增，n 仍为 2，s 的值为 sizeof(int)=4，类型为 size_t。不能把 sizeof 中的表达式当作普通运行表达式。

题 6 解析

先调用 add，得到 7；更改目标后调用 multiply，得到 10。(*op) 中括号使 * 与 op 组成指针声明；后面的 (int,int) 描述目标函数的参数类型。去掉前一对括号，声明结构会变成返回指针的函数，而不是函数指针变量。

题 7 解析

第一段显示 1a 10 16：hex 对后续整数持续生效，直到 dec 恢复十进制。第二段显示 007 8：setw 的宽度用于下一次相关输出，之后被重置；填充字符保留，但没有宽度要求的 8 不会凭空补零。

题 8 解析

四句都不能作为无条件正确结论。bool 和指针大小由实现条件决定；有效地址空间与物理内存不是同一个量；在给定 32 位 unsigned int 条件下，移位量等于位宽属于未定义行为，不可指定结果 0。解题时首先标清环境假设与语言合法性。

把补漏题归入四个步骤

先识别表达式类型和语境；再写位宽、编码、寻址单位等已知条件；解析运算结构并逐步计算；最后检查单位、边界和显示格式。缺少条件的题目应给条件式答案，而不是拿某一次机器输出冒充所有环境的结论。

旧卷与课件练习详细解析

B1. E22 Q14 (第 3 页) : 40 位字节寻址

原题给定地址是 40 位、最小可寻址单位是 8 位字节，且 $1\text{KB}=2^{10}$ 字节、 $1\text{MB}=2^{10}$ KB、 $1\text{GB}=2^{10}$ MB。问理论管理容量；选项为 40、256、320、1024 GB。

先有 2^{40} 个地址，每地址对应一个字节，因此总容量 2^{40} 字节。按题面的二进制换算， $1\text{GB}=2^{30}$ 字节，故 $2^{40}/2^{30}=2^{10}=1024\text{GB}$ ，选 **d**。这不是把 40 乘某个换算因子，而是用地址位数数出地址组合。

现代单位记法会用 GiB 表示 2^{30} 字节，但答原题按它明确规定的 GB 解释。地址从零开始不会让容量少一字节；最大地址编号与地址数量不同。

B2. E22 Q17 (第 4 页) : 提取 αRGB 的哪一字节?

原代码：

```
int pixel = 0x05192022;
int color = (pixel >> 8) & 0xFF;
```

原题按四个字节解释为 α 、Red、Green、Blue。分组得到 05、19、20、22。最右边的蓝色 22 占低八位；右移八位，把原来的 Green=0x20 移到最低字节；与 0xFF 相与只保留这八位。

```
原值：      05 19 20 22
右移 8 位： 00 05 19 20
保留低字节：      20
```

$0x20=2\times 16+0=32$ ，答案 **Green、32、选 d**。易错答案 20 是直接把十六进制数字串当成十进制；另一种错误是少考虑右移而选 Blue。本例按题设的足够宽 int、正整数进行，不扩展负有符号数的右移讨论。

教学变式：若改为 $(\text{pixel} \gg 16) \& 0xFF$ ，取 Red=0x19=25；若改为 $\text{pixel} \& 0xFF$ ，取 Blue=0x22=34。每次先标目标字节位置，再移动和掩码。

B3. E22 Q20 (第 5 页) : double (*f)(double)

问声明含义，选项包括“返回 double* 的函数”“函数对象”“指向一个 double 参数且返回 double 的函数的指针”等。答案 **c：函数指针**。

从 f 向外：(*f) 使 f 是指针；(double) 是目标函数参数列表；最左侧 double 是目标函数返回类型。double* f(double) 才是接收 double、返回 double* 的函数声明。括号改变声明的结构，并非可有可无的排版。

B4. E22 Q11 (第 3 页) : 回扣 M1 的类型与分组

原表达式：

```
9.0*(8-7) / 6 + 5*4 % 3*(2+1)
```

括号先得到 1 和 3。左半边 9.0×1 为 double，再除以 6 得 1.5。右半边的 *、%、* 同级左结合： $5\times 4=20$ ， $20\%3=2$ ， $2\times 3=6$ 。最后 double 与 int 相加得到 7.5，选 **d**。

% 只用在右半边的整数子表达式，不因另一半出现 9.0 就让所有局部运算都提前变成 double。相反，若把 9.0 改成整数 9，左半边将进行整数除法得 1，最终为 7。只改一个字面量就改变中间类型，这是最小对照。

B5. L5 格式变式：四位十六进制与最小宽度

从新流开始，写 `out << uppercase << hex << setfill('0') << setw(4) << 17 << ' ' << 19;`，显示 **0011 13**。17 按 hex 为 11，宽四位所以补两个零；19 按 hex 为 13，但 `setw` 的宽度已经重置，不再自动补四位。

若想每个数都四位，第二个整数前也要 `setw(4)`。`setw` 是最小宽度，遇到 0x12345 这类五位数时不会把它裁成四位。这与 Q32 要求的答案格式相连；Q32 的数值项和地址项须先分别算出，不能直接把每个表达式塞进同一种打印语句。

B6. 六册之后，旧卷哪些内容仍然需要学？

旧卷位置	尚需的独立模块	现在已有的前置
Q8、Q9、Q23	哈希、迭代器、容器与容器适配器	对象、尾后边界、流式接口
Q10、Q21	编程范式与 OOP 完整比较	本册只补了部分术语
Q13、Q18、Q19	复杂度、归并排序、BST 与堆	数组、对象、基本函数模型
Q16、Q27	minimax、递归与回溯	参数调用、状态与终止判断
Q28	环形队列及 head/tail 约定	数组容量、下标、模运算
Q29、Q30 深入部分	复制资源、继承、多态的完整设计	M7 已解释原题与入门边界

本次找到的七份课件是 C++ 基础材料，L1 第 13 张本来就还列有后续数据结构与算法课程。不能仅因后续课件暂未提供，就宣布这部分不考或给它们编造课件页码。六册完成的是当前这一段基础及相应旧卷题，不替代整门课的后续学习。